

Towards Robust Newton’s Method Optimization using Evolutionary Strategies

Sudhan Chitgopkar and Michael Zhan
Harvard University
(Dated: April 29, 2026)

Optimization is a prominent topic within applied mathematics and has vast applications in a wide variety of fields. There are no universal optimization algorithms, and all methods used today have advantages and disadvantages. In particular, we seek to combine the high-level approach of evolutionary strategies (ES) with the quadratic convergence and local robustness of Newton’s method to develop a robust hybrid approach. Here, we develop and test this approach against other common optimization methods on a series of benchmarks. We discover that these hybrid methods are less sensitive than Newton’s method to initial conditions yet reach higher accuracies than ES alone, at the slight expense of more iterations and run time. Though optimization algorithms are objective function-dependent, we are confident that the general trend holds for our hybrid method.

I. BACKGROUND

Optimization is one of the most prominent topics in the field of applied math and holds applications throughout a wide variety of industries. In particular, we concern ourselves with unconstrained optimization, of the form

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad (1)$$

to optimize some input vector $\mathbf{x}$ on the objective function $f[4]$.

Perhaps one of the most well-known optimization algorithms is a modification of the Newton-Raphson method and its quadratic convergence, based on the first and second derivatives of the objective function. Many other optimization algorithms have been developed, including the family of evolutionary strategies. While Newton’s method tackles a problem based on its intimate knowledge of the objective function, evolutionary strategies optimize in a way simulating nature, pruning a population based on fitness.

Despite the fame of Newton’s method, the optimization algorithm has shortcomings, particularly in its sensitivity to initial conditions. In this paper, we aim to combine evolutionary strategies with Newton’s method to develop a more robust hybrid optimization method that has less sensitivity to initial conditions, yet still retains the advantages of Newton’s method.

A. Motivation

Though the Newton-Raphson method is a widely-used and straightforward method to solve optimization problems, it suffers from its dependency on an initial guess proximate to the optimum. Performing Newton’s method on a weak initial point can lead to a myriad of undesirable behavior including termination due to a stationary point, entering into an infinite cycle, divergence due to the function’s nature, or even undefined results. Even more problematically, slight perturbations in an initial

guess can result in the algorithm converging on wildly different minima, especially in functions with multiple local minima. Despite these shortcomings, Newton’s method is an otherwise-fast optimization algorithm capable of quadratic convergence.

To this end, we propose a hybrid method combining Evolutionary Strategies (ES) with Newton’s method.

While ES and genetic algorithms (GAs) as a whole may be slower and more computationally-expensive than other function-based optimization algorithms due to their stochastic and simulational nature, they require no additional information outside of the function itself and have less dependence on initial conditions. In contrast, Newton’s method for optimization requires either analytical or numerical calculations of the gradient and the Hessian, potentially resulting in significant overhead and poor run-time. Our hybrid method aims to combine the advantages of each method, with the initial wide-scope search performed by the less sensitive method ES, and the latter fine-tuned optimization performed by the locally-robust Newton’s method.

B. Prior Work

The idea of using a Genetic Algorithm (GA) in conjunction with Newton’s method is not entirely novel — such “hybrid” algorithms have been utilized for applications ranging from civil engineering[1] to artificial intelligence[8]. However, the vast majority of the literature investigating such hybrid approaches tune them to very specific tasks, like asteroid exploration[5]. Certainly, more general approaches have been designed for global optimisation[6], though there is little metadiscourse about how such algorithms *should* work, their optimal parameters, their effectiveness relative to other optimization techniques, or even *which* GAs tend to be effective. Indeed, the literature surveys hybrid algorithms which use everything from standard genetic algorithms to competitive co-evolution with few comparisons to effectiveness relative to other optimization techniques or

even the standalone techniques which comprise the hybrid approach.

In addition to proposing a new design for a hybrid GA-Newton method, this paper seeks to bridge a gap in the literature by these newly designed algorithms to “off-the-shelf” optimization algorithms as well as non-hybrid approaches using just genetic programming and Newton’s method.

II. DESIGN

We propose two approaches to a hybrid GA-Newton’s method for global optimization. In each, an Evolutionary Strategies (ES) approach will be used to search some function for its global optimum, and converge to a strong initial point for use with Newton’s method. Together, these algorithms are expected to provide a fast and robust method by which to find the global minimum of a function. In order to ensure that the ES and Newton’s method are being used to their strengths, we design the ES to terminate after it is within a certain “broader” threshold of the solution accuracy to ensure that we aren’t using the ES’ slower, stochastic, black-box searching when we could be using Newton’s fast convergence. Once our algorithm switches to using Newton’s method, the threshold for solution accuracy becomes stronger, allowing us to use Newton’s method to find as accurate a solution as possible. The only difference between the two approaches we use here will be the *type* of ES originally employed. In our first SIMPLEES algorithm, we will use an EA whose exploratory component remains relatively constant and leads to slower, more stable convergence. In our second CMA-ES, we will use an EA with a dynamic exploratory component which requires, on average, fewer iterations to find suitable initial points but is somewhat unstable in the process.

This approach is distinct from many existing approaches in three ways. Firstly, many existing hybrid approaches use Newton’s method to determine the fitness of individuals in a genetic algorithm, which is then run until it reaches the global optimum[6]. In contrast, we propose avoiding use of Newton’s method until the our ES has terminated, thereby minimizing the amount of gradient and Hessian computations completed and allowing for quite computationally cheap iterations of the ES.

This, too, is not entirely novel as a this approach was used by Yang (2021) in testing the parallel performance of such approaches[9]. However, Yang tests *specifically* with Genetic Algorithms while we propose the use of Evolutionary Strategies — a different paradigm of genetic programming. Because ES approaches are specifically designed with optimization in mind, we expect this to be a more accurate and efficient approach than others proposed in the literature.

The final unique characteristic of our design is the penalization of crossover potential of individuals in our ES if they have characteristics which would make them fail as an initial value to Newton’s method *even if* they’re especially close to a minimum. This, we hope, will make our design more cohesive — the ES should work *for* Newton’s method, not just *before* Newton’s method.

III. IMPLEMENTATION

A. Newton’s Method

Here, we follow a standard implementation for Newton’s method with the following iterative step.

$$\Delta f(\mathbf{x}_k) \mathbf{s}_k = -\nabla f(\mathbf{x}_k) \quad (2)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{s}_k \quad (3)$$

The Hessian and gradient of the given function f are calculated numerically using the `numdifftools` Python package using the following methods.

```
def gradient(x):
    return nd.Gradient(f)(x)

def hessian(x):
    return nd.Hessian(f)(x)
```

Both are determined using finite-difference methods, and mixed partial derivatives are improved using Richardson extrapolation. With the gradient and Hessian defined as above, we use Equation 3 to code Newton’s method as follows.

```
def newton(f, x0, tol=1e-10, maxiter=1000):
    count = 0
    x = x0
    prev = x0
    for i in range(maxiter):
        count += 1
        x = x - np.linalg.pinv(hessian(x)) @ gradient(x)
        if np.linalg.norm(x - prev) < tol:
            break
        prev = x
    else:
        print(f'Failed to reach tolerance {tol}'
              'in {maxiter} iterations.')
    return x, count
```

This specific implementation of Newton’s method allows for a custom tolerance (`tol`) and maximum number of iterations (`maxiter`) to be set. To ensure numerical accuracy in optimizing on the input vector x , we stop on the condition $|\mathbf{x}_{k+1} - \mathbf{x}_k| < \text{tol}$.

B. Evolutionary Strategies

Here, we implement two different Evolutionary Strategies (ES) approaches: (1) SIMPLEES and (2) CMA-ES.

1. SIMPLEES

Our SIMPLEES uses, as the name implies, a relatively naïve ES to optimize some function. At its core, SIMPLEES consists of three components: **Agent**, **Population**, and **Evolution**. Each **Agent** is a candidate solution which consists of three components:

- **id** — a unique identifier for each agent
- **x** — a candidate solution to the minimization
- **fitness** — $f(\mathbf{x})$ for minimization function, f

where we seek to *minimize* an **Agent**’s fitness. Each **Population** consists of a list of **Agents**, and represents the amalgam of candidate solutions at some iteration of the **Evolution**. Finally, the **Evolution** manages the **Population** (and **Agents** therein) by determining which **Agents** in the current **Population** to use for future **Populations**, and generating those **Populations** until the **Agents** in them converge upon a candidate solution. We show the pseudocode for this algorithm in Algorithm 1.

Algorithm 1 SIMPLEES

```

1: procedure EVOLVE( $x_0$ )
2:    $generations = 0 \leftarrow$  Generations evolved
3:    $N \leftarrow$  number of agents in population
4:    $pmean = x_0 \leftarrow$  mean of prev. generation
5:    $mean = x_0 \leftarrow$  initial mean
6:    $stddev = 1 \leftarrow$  initial  $\sigma$ 
7:    $pop = population(N, gauss(mean, stddev))$ 
8:
9:   while  $generations < 100$ 
10:     $nextPop = pop.getTop(k)$ 
11:
12:    if  $generations$  divisible by 5 then
13:      for agent in  $nextPop$  do
14:        if agent has invalid Hessian then
15:           $nextPop.remove(agent)$ 
16:
17:       $pmean = mean$ 
18:       $mean = nextPop.getMean()$ 
19:       $stddev = nextPop.getStddev()$ 
20:       $pop = nextPop +$ 
         $population(N - k, gauss(mean, stddev))$ 
21:
22:      if  $norm(mean - pmean) < tolerance$  then
23:        end
24:      else
25:         $generations = generations + 1$ 
26:        goto while.
```

For each generation, the k **Agents** with the lowest fitness (recall, lower is better) values are chosen, and their

mean and standard deviation are calculated. This pool makes up the “parents” whose genes (i.e. the standard deviation and mean) are used to seed the next generation of “children”. Both the parents and their children now compose the next generation’s population. This process repeats until *either* we’ve reached some maximum amount of pre-specified iterations or until the difference between means between generations reaches a pre-specified tolerance.

Every 5 generations, each of the parents’ Hessians are checked to ensure that they are valid initial inputs to Newton’s method (i.e. defined for the Newton’s method calculations). The list of parent’s is pruned of **Agents** with invalid Hessians to ensure that their “genes” (candidate solutions) do not propagate to future generations. This is done *only* for parents every 5 rounds because the Hessian calculations require taking second derivatives and are thus computationally expensive.

2. CMA-ES

CMA-ES is a modification to SIMPLEES using a covariance matrix, which allows the ES to better learn the objective function f , especially in cases where f is ill-conditioned. While SIMPLEES samples from an assumed independent multivariate normal random distribution in each generation, CMA-ES aims to update the covariance matrix of the multivariate distribution to capture any co-dependencies between dimensions in the objective function. Additionally, CMA-ES tracks its evolution paths throughout the algorithm, allowing it to search for the optimum faster in favorable directions and preventing premature convergence[3].

Within CMA-ES, we primarily track the covariance matrix C and step size σ , as the multivariate distribution parameter becomes

$$x_i \sim \mathcal{N}(\mu, \sigma^2 C) \sim \mu + \sigma \mathcal{N}(0, C) \quad (4)$$

where μ is the mean. Here, σ is the step size and represents the scale of the distribution. Apart from C and σ , however, we also track the evolution of each of the two, allowing CMA-ES to achieve its favorable properties. Specifically, we track the evolution paths p_C and p_σ , representing the evolution paths for C and σ , respectively.

For the most part, CMA-ES follows the same logical flow as SIMPLEES, creating a **Population** of **Agents** out of which λ parents are chosen to seed the next generation. However, CMA-ES takes extra steps to update the step size σ and the covariance matrix C .

For each generation, we first generate the **Population** with **Agents** by sampling the multivariate normal distribution.

$$x_i^{(t+1)} \sim \mu^{(t+1)} + \sigma^{(t+1)} \mathcal{N}(0, C^{(t+1)}) \quad (5)$$

From this **Population**, we then choose the λ most fit **Agents** as parents for the next generation.

We then proceed to work to update σ and C . We begin by finding the new mean

$$\mu^{(t+1)} = \mu^{(t)} + \alpha_\mu \frac{1}{\lambda} \sum_{(i=1)}^{\lambda} (x_i^{(t+1)} - \mu^{(t)}) \quad (6)$$

where α_μ is a learning rate for the mean, typically set to 1.

We then update the two evolution paths, which are necessary to later update σ and C . We have

$$p_\sigma^{(t+1)} = (1 - \alpha_\sigma) p_\sigma^{(t)} + \sqrt{\alpha_\sigma(2 - \alpha_\sigma)} \lambda C^{(t)-\frac{1}{2}} \frac{\mu^{(t+1)} - \mu^{(t)}}{\sigma^{(t)}} \quad (7)$$

$$p_c^{(t+1)} = (1 - \alpha_c) p_c^{(t)} + \sqrt{\alpha_c(2 - \alpha_c)} \lambda \frac{\mu^{(t+1)} - \mu^{(t)}}{\sigma^{(t)}} \quad (8)$$

where α_σ and α_c are learning rates for their respective evolution paths. Notably, because C is a covariance matrix in a multivariate Gaussian distribution, it is real and positive semi-definite. Thus, it is always diagonalizable, its eigenvalues are real and non-negative, and all of its eigenvectors are orthogonal[2]. We can represent C as a decomposition

$$C = B^T D^2 B$$

where B is an orthogonal matrix of C 's eigenvectors and D^2 is a diagonal matrix of real and non-negative eigenvalues. Since D^2 is real and non-negative, we have

$$C^{\frac{1}{2}} = B^T D B$$

and $C^{\frac{1}{2}}$ is invertible.

We finally update C and σ . We can estimate a new C based on two factors: the history of C and the chosen elite. The history of C is represented by p_C , the path of evolution, and can thus be used to make a rank-one approximation of the new C . Alternatively, the λ members of the chosen elite (those acting as ‘‘parents’’ for the next generation) can be used to estimate the covariance matrix as well. Thus, we have

$$C^{(t+1)} = (1 - \alpha_1 - \alpha_\lambda) C^{(t)} + \alpha_1 p_C^{(t+1)} p_C^{(t+1)T} + \alpha_\lambda \frac{1}{\lambda} \sum_{i=1}^{\lambda} \frac{(x_i^{(t+1)} - \mu^{(t)})(x_i^{(t+1)} - \mu^{(t)})^T}{\sigma^{(t)2}} \quad (9)$$

$$\sigma^{(t+1)} = \sigma^{(t)} \exp \left(\alpha_\sigma \left(\frac{\|p_\sigma^{(t+1)}\|}{\mathbb{E}\|\mathcal{N}(0, I)\|} - 1 \right) \right) \quad (10)$$

where α_1 and α_λ are the learning rates for the two approximations of C , and α_σ is the learning rate for σ .

With the new covariance matrix C and step size σ , we begin the next iteration of CMA-ES, creating a new

Population from the new mean, covariance matrix, and step size.

We use a stopping condition based on the Euclidean distance between some previous mean $\mu^{(t)}$ and the new mean $\mu^{(t+1)}$. Our goal with this optimization is to seek the vector $\mathbf{x}^*$ that minimizes the objective function, so we iterate until the input vector $\mathbf{x}$ no longer changes by more than some tolerance. At this point, we exit the optimization loop and conclude CMA-ES.

Algorithm 2 CMA-ES

```

1: procedure EVOLVE( $x_0$ )
2:    $a.\mu, a.\sigma, a.c, a.\lambda, a.1 \leftarrow$  learning rates
3:    $N \leftarrow$  number of agents in population
4:    $\lambda \leftarrow$  number of parents per generation
5:    $mean \leftarrow$  initial mean
6:    $time = 0 \leftarrow$  generation
7:    $\sigma = 1 \leftarrow$  initial step size
8:    $cov = identity \leftarrow$  initial covariance
9:    $p.\sigma = 0 \leftarrow$  initial evolution path for sigma
10:   $p.c = 0 \leftarrow$  initial evolution path for cov
11:   $pop = population(N, gauss(mean, cov))$ 
12:  while  $time < max\_iter$ 
13:     $nextPop = pop.getTop(\lambda)$ 
14:    if  $time \bmod 5 == 0$  then
15:      for  $agent$  in  $nextPop$  do
16:        if  $agent$  has near-zero Hessian then
17:           $nextPop.remove(agent)$ 
18:       $prev\_mean = mean$ 
19:      Update  $mean$  following equation 6
20:      Update  $p.\sigma$  following equation 7
21:      Update  $p.c$  following equation 8
22:      Update  $cov$  following equation 9
23:      Update  $\sigma$  following equation 10
24:       $pop = nextPop + population(N - \lambda, gauss(mean, cov))$ 
25:       $time = time + 1$ 
26:      if  $norm(mean - prev\_mean) < tolerance$  then
27:        end
28:      else
29:        goto loop

```

Similarly to SIMPLEES, every 5 generations, we numerically audit the Hessian of each of the λ parents to ensure that they are non-zero and can thus be used in our hybrid method. In our implementation, we choose $\alpha_\sigma = 0.5, \alpha_c = 1 = \alpha_\mu = 1, \alpha_1 = \alpha_\lambda = 0.3$, and initialize step size $\sigma = 1$.

3. HYBRID METHODS

Our implementation of hybrid methods includes two versions: a combination of SIMPLEES with Newton's method, and a combination of CMA-ES with Newton's method. In both, we first use the evolutionary strategy on a given input vector $\mathbf{x}_0$ to a tolerance more lenient than our final tolerance, then input the returned partially-optimized vector into Newton's method with the desired strict tolerance.

IV. METHODS

We seek to compare the following algorithms against one another with respect to their ability to optimize some given functions.

- Newton’s method
- SIMPLEES
- CMA-ES
- SIMPLEES-Newton hybrid
- CMA-ES-Newton hybrid
- An “off-the-shelf” optimization method

Each of these algorithms will be compared across the following parameters

- Accuracy of minimum found
- Run-time
- Iterations taken
- Tolerance to poor starting conditions

To this end, we will begin by coding our own implementations, from scratch, for Newton’s method and each of the ES/hybrid approaches above-described. Due to its prolific use in the Python and data science communities, we will be comparing each of these optimization algorithms against `scipy.optimize`. To ensure that the optimization problems solved during testing are non-trivial, we will use the following two-dimensional standard performance test problems:

- Cigar
- Ackley
- Bohachevsky
- Griewank
- Rosenbrock

Altogether, we expect this to give us robust and comprehensive results about the benefits and shortcomings of each approach amidst a variety of conditions.

V. RESULTS

We begin by exploring the effectiveness of each algorithm at finding the global minimum of a function. Figure 1 depicts the Euclidean distance between the results of each algorithm and the actual global minimum across the test functions aforementioned. Though there is variance between each algorithm’s ability to find the global optimum with respect to the function it is run on, this is somewhat expected. The Cigar function is a very smooth polynomial function especially in two dimensions, and is thus an objective function where Newton’s method is expected to perform very well because there are no local minima for initial values to get “stuck” in.

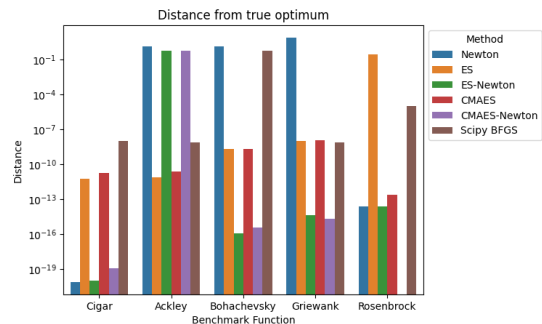


FIG. 1. Distance from each calculated optima $\mathbf{x}^*$ and the known true optimum over benchmark objective functions.

Indeed, we see that the base Newton’s method as well as the two hybrid Newton’s method algorithms achieve solutions well within a distance 10^{-16} of the intended optimum of $(0,0)$, whereas the higher-level evolutionary strategy approaches achieve solutions only within 10^{-10} . We see a similar result with the off-the-shelf `scipy` algorithm with minimal tuning.

Ackley’s function in Figure 2, on the other hand, is jagged with many local minima for optimization algorithms to fall into. We see the result of this instantly with Newton’s method, where the algorithm converges to a local minimum away from the desired minimum at $(0,0)$. Notably, even when the ES-component of our hybrid algorithms pass a “close-enough” initial point (per our threshold) and pass off computation to Newton’s method, the algorithms still converge on a different minimum. This is due to the nature of the Ackley’s function, where the rapidly changing gradient and Hessian results in erratic Newton’s method behavior. Because neither ES nor CMA-ES rely on intimate function knowledge, the evolutionary strategy algorithms are able to find solutions essentially at the desired optimum, with errors on the magnitude of 10^{-10} .

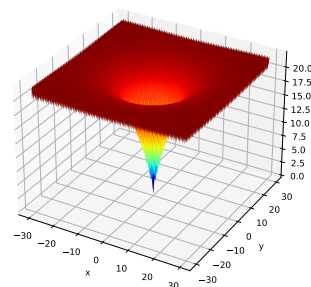


FIG. 2. Ackley’s function

The Bohachevsky and Griewank functions are two examples which demonstrate the power of the hybrid method. For both, basic Newton’s method fails, with a given initial point of $(1,1)$, to converge to the desired optima of $(0,0)$. We see that SIMPLEES and CMA-ES alone are able to achieve solutions correct to around 10^{-9} ,

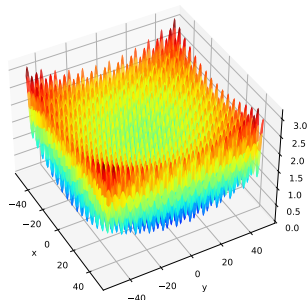


FIG. 3. Griewank's function

but our hybrid methods are able to achieve significantly better solutions which are correct to the order of 10^{-16} . Our hybrid algorithm results on these functions thus exemplify our original goal of using evolutionary strategy algorithms to overcome the dependence of Newton's method on initial points, as we are essentially widening the initial point neighborhood where our optimization algorithm converges, without sacrificing accuracy. Notably, the Griewank function, represented in Figure 3, boasts a similar jagged nature to Ackley's function, which (in Ackley's function) caused the pure Newton's method approach to converge on an undesired optimum. However, perhaps due to the increased width of the peaks, our hybrid methods successfully converged here, while they

did not for Ackley's function. This begins to suggest that our hybrid method, like many optimization methods, is not universal and that its performance is instead function-dependent.

Finally, we consider the Rosenbrock function — another optimization benchmark function with a minimum at $(1, 1)$. Interestingly, we observe an opposite trend here to the other benchmark functions. ES is unable to converge exactly to $(1, 1)$, while the other functions do converge to within 10^{-13} . In addition to further demonstrating the function-dependence of convergence with the algorithms presented, the Rosenbrock function is also curious because of the difference in effectiveness of the SIMPLEES and CMA-ES, which has been otherwise low. To further investigate this, we can visualize and track the Populations that each algorithm generates over iterations and compare them. Figure 4 depicts the population plots for SIMPLEES at three early iterations. Notably, we see that SIMPLEES converges quite quickly which would normally be desirable, except that it converges on an *incorrect* minimum. In contrast, consider the population plots for CMA-ES, depicted in Figure 5. Here, we see much slower convergence and a highly exploratory population, even at iteration 12 when SIMPLEES has converged. This *is* desirable behavior for functions like Griewank, even in contrast to similar functions like Ackley's because there is not an abundantly clear global minimum, even as the ES approaches it.

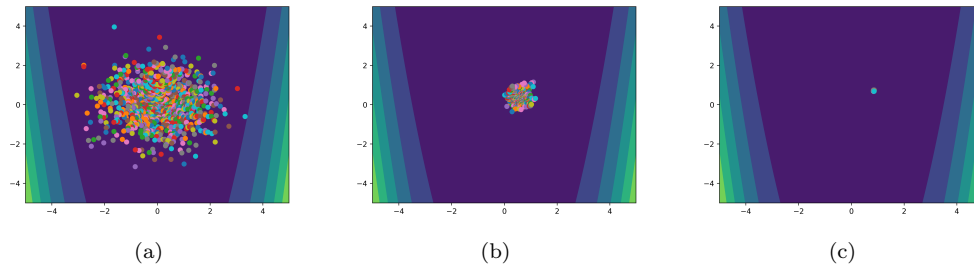


FIG. 4. Population plots for SIMPLEES at generations 0(a), 7(b), and 12(c) for the Rosenbrock function

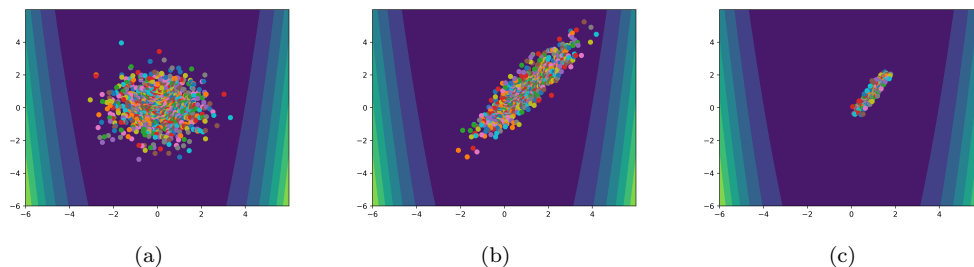


FIG. 5. Population plots for CMA-ES at generations 0(a), 7(b), and 12(c) for the Rosenbrock function

We now move to comparing the number of iterations (Figure 6) and the run time (Figure 7) of each method for each benchmark, and see that a clear trend arises. Because evolutionary strategies use a simulational approach, we expect them to require more iterations than a direct approach such as Newton’s method. We see that SIMPLEES and CMA-ES indeed each require more iterations than Newton’s method, and that the hybrid methods (which combine both evolutionary strategy and Newton’s method), require more than their respective individual algorithms (and so they should, as they are a combination of both individual algorithms). Of note, we see function variance here too. For example, the hybrid algorithms on Ackley’s function require a large number of iterations, stemming from the erratic behavior of Newton’s method on this function. Similarly, Newton’s method converges to an undesired optimum on the Bohachevsky function, and fails to converge at all on the Griewank function. The run time trend was similarly uniform, and

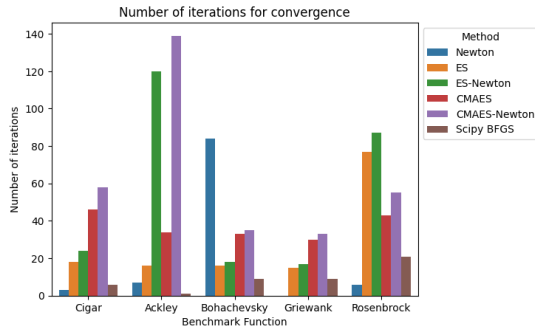


FIG. 6. Number of iterations required for methods to converge. Note that Newton’s method did not converge on the Griewank function, so its iteration count has been omitted.

was generally proportional to iterations, with Newton’s method performing the fastest (if it converged), followed by SIMPLEES and its corresponding hybrid, then CMA-ES and its hybrid. Finally, we evaluate the sensitivity

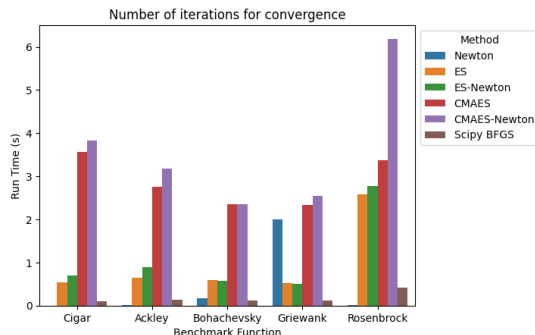


FIG. 7. Run time for each method.

of our optimization algorithms to initial points. Using the Griewank function with desired optimum $(0,0)$, we initialized our algorithms at points (x,x) for $x \in [0,2]$,

shown in Figure 8. From these results, we note the relative accuracy and the relative sensitivity of the optimization algorithms to initial points. As described above and shown in Figure 8, Newton’s method starts off as the most effective algorithm when provided an initial point extremely close to the true global minimum. By contrast SIMPLEES and CMA-ES perform quite poorly, and the hybrid methods perform much better than their ES-only counterparts but not as well as Newton’s method. `scipy` performs at the level of Newton’s method (likely because it actually uses Newton’s method when close enough to the minimum). As the initial value’s distance from the global minimum increases, though, Newton’s method’s accuracy decreases sharply leading to a much higher error rate. The same is true of the `scipy` algorithm, though it’s error rate caps out around the level of effectiveness of the ES approaches. As we move even further from the initial value, the hybrid approaches are the clear winner in terms of result accuracy, followed by `scipy` and the ES approaches, with Newton’s method being entirely unusable. Finally, even the SIMPLEES-Newton approach fails to converge while the CMA-ES-Newton method holds strong.

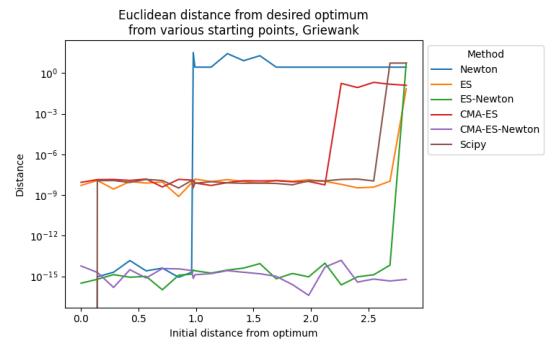


FIG. 8. Distances from calculated optima to desired optimum for the Griewank function, from an initial point with increasing distance from the desired optimum.

VI. CONCLUSION

Here, we have designed and implemented two somewhat novel algorithms for global optimization which rely on a hybrid between Evolution Strategies and Newton’s Method. At a high level, each algorithm uses an ES to search a function for suitable initial points (penalizing candidate solutions which would make Newton’s method fail), then passes off computation to Newton’s method as the ES converges. We explore this approach using the SIMPLEES and the CMA-ES where the latter is more exploratory than the former.

We find that when compared solely to “off-the-shelf” optimization algorithms, an ES-only, and a Newton’s-Method-only approach, our hybrid algorithms tend to perform better across functions which are jagged and dif-

difficult to optimize using simple methods. For Griewank's function specifically, SIMPLEES and CMA-ES were able to find the global minimum within a reasonable accuracy with *far worse* initial values than any other method and remained competitive *even when* good initial values were provided.

This performance, however, comes at a cost. SIMPLEES and CMA-ES consistently ran for longer and required more iterations than any of its counterparts across almost all test functions. For simple functions like the Cigar test function, or when the initial value is very close to the true solution, each hybrid approach also performs (on average) an order of magnitude worse than just Newton's method. In general, though, we argue that the hybrid approach presented here is a versatile optimization approach when there is little information about the function itself and good initial values.

There are also somewhat different use-cases for SIMPLEES and CMA-ES. The latter takes longer to converge but tends to be more dynamic with how exploratory it is, making it suitable for unknown functions or functions which are known to be difficult to optimize. On the other hand, SIMPLEES provides performance which generally rivals that of CMA-ES while being more resource conscious — taking fewer iterations and less run-time.

Importantly, this paper has only considered the performance of these algorithms on two-dimensional optimization problems. As the number of dimensions increases, ES approaches may continue to be somewhat useful, especially as their pervasiveness in machine learning applications increases[7], but this is not yet clear. Future work in this area might include running tests similar to the ones shown here on problems of higher dimensions and analyzing their effectiveness to other “off-the-shelf” algorithms more suited to high-dimensional optimization.

REFERENCES

- [1] Chuen-Jyh Chen. “An Integrating Genetic Algorithm and Modified Newton Method for Tracking Control and Vibration Suppression”. In: *Artif. Intell. Rev.* 53.5 (June 2020), pp. 3177–3199. ISSN: 0269-2821. DOI: 10.1007/s10462-019-09759-8. URL: <https://doi.org/10.1007/s10462-019-09759-8>.
- [2] Robert M Freund. *Symmetric Matrices and Eigendecomposition*. 2014.
- [3] Nikolaus Hansen. *The CMA Evolution Strategy: A Tutorial*. 2023. arXiv: 1604.00772 [cs.LG].
- [4] M.T. Heath. *Scientific Computing: An Introductory Survey*. McGraw-Hill Education, 2005. ISBN: 9780071244893. URL: <http://books.google.de/books?id=gwBrMAEACAAJ>.
- [5] Ryuji Koshikawa, Akira Terui, and Masahiko Mikawa. *Solving System of Nonlinear Equations with the Genetic Algorithm and Newton's Method*. 2020. arXiv: 2007.05159 [math.NA].

- [6] Marcus M. Noack and Simon W. Funke. “Hybrid genetic deflated Newton method for global optimisation”. In: *Journal of Computational and Applied Mathematics* 325 (2017), pp. 97–112. ISSN: 0377-0427. DOI: <https://doi.org/10.1016/j.cam.2017.04.047>. URL: <https://www.sciencedirect.com/science/article/pii/S037704271730225X>.
- [7] Masahiro Nomura and Isao Ono. “Fast Moving Natural Evolution Strategy for High-Dimensional Problems”. In: *CoRR* abs/2201.11422 (2022). arXiv: 2201.11422. URL: <https://arxiv.org/abs/2201.11422>.
- [8] Yunfeng Wang et al. “A Hybrid Computational Intelligence Method of Newton's Method and Genetic Algorithm for Solving Compatible Nonlinear Equations”. In: *Applied Mathematics and Nonlinear Sciences* 8.1 (2023), pp. 1731–1742. DOI: [doi:10.2478/amns.2022.2.0161](https://doi.org/10.2478/amns.2022.2.0161). URL: <https://doi.org/10.2478/amns.2022.2.0161>.
- [9] Yunwen Yang. “Parallel performance analysis of nonlinear equations algorithm based on hybrid genetic algorithm”. In: *Journal of Physics: Conference Series* 1982 (July 2021), p. 012108. DOI: [10.1088/1742-6596/1982/1/012108](https://doi.org/10.1088/1742-6596/1982/1/012108).

Appendix A: Performance Test Functions

1. Cigar

$$f(x) = x_0^2 + 10^6 \sum_{i=0}^2 x_i^2$$

2. Ackley

$$f(\mathbf{x}) = 20 - 20 \exp \left(-0.2 \sqrt{0.5 \sum_{i=0}^2 x_i^2} \right) + e - \exp \left(\sqrt{0.5 \sum_{i=0}^2 \cos(2\pi x_i)} \right)$$

3. Bohachevsky

$$f(\mathbf{x}) = \sum_{i=1}^1 (x_i^2 + 2x_{i+1}^2 - 0.3 \cos(3\pi x_i) - 0.4 \cos(4\pi x_{i+1}) + 0.7)$$

4. Griewank

$$f(\mathbf{x}) = 1 + \frac{1}{4000} \sum_{i=1}^2 x_i^2 - \prod_{i=1}^2 \cos \left(\frac{x_i}{\sqrt{i}} \right)$$

5. Rosenbrock

$$f(\mathbf{x}) = \sum_{i=1}^1 [(1 - x_i)^2 + 100(x_{i+1} - x_i^2)^2]$$

Appendix B: Source Code

All source code used for this project can be found [here](#).
